

Task 9 —: Implement Role-Based AWS Access for a Development Team

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The goal of this task was to design an IAM access structure for different members of a small development team, which included a Developer, QA Engineer, Operations Engineer, and an HR/Admin User. Each person needed different levels of AWS access based on their job responsibilities.

My approach was to strictly follow the principle of least privilege. Instead of creating one broad policy, I created separate policies for each role, ensuring that each user only had the permissions necessary to perform their specific job. I started by defining the responsibilities of each role, identified the AWS resources they needed, and created an access matrix to map responsibilities to required actions. Then, I designed custom IAM policies, created users, attached the policies, and tested both permitted and denied actions to validate the access model.

2. Implementation Steps

Here are the steps I followed to complete this task:

Step 1: I first defined the responsibilities of each role. The Developer is responsible for writing code and deploying to the dev environment. The QA Engineer tests applications and manages the test environment. The Operations Engineer manages production, monitors resources, and handles incidents. The HR/Admin User manages IAM users and views billing.

Step 2: I identified the AWS resources each role needed. The Developer needs access to EC2 (dev) and S3 (dev). The QA Engineer needs

access to EC2 (test) and S3 (test). The Operations Engineer needs access to EC2 (prod), S3 (prod), and RDS. The HR/Admin User needs access to IAM user management and billing.

Step 3: I built an access matrix that mapped each permission to the appropriate role. This helped me visualize what each user should and should not be able to do before I started writing policies.

Step 4: I designed IAM policies for each role. I created DevPolicy, QAPolicy, OpsPolicy, and HRPolicy with specific actions and resources.

Step 5: I granted only the required permissions by attaching each policy to its respective user.

Step 6: I tested permitted actions by running commands like `aws ec2 describe-instances` for both the Developer and QA users.

Step 7: I tested denied actions by trying to start EC2 instances. Both Developer and QA users received AccessDenied errors, confirming that the policies were working correctly.

Step 8: I reviewed all permissions to ensure there was no excessive access.

Step 9: I documented the reasoning behind each permission, explaining why each action was granted.

Step 10: I identified that temporary or restricted access could be handled using AWS STS for emergency situations.

Step 11: I explained why sharing one administrator account is inappropriate, mainly because it violates security best practices and eliminates accountability.

Step 12: I documented credential-management best practices, such as enabling MFA, rotating access keys, and using CloudTrail for auditing.

Step 13: I reviewed the final IAM structure to ensure it aligned with the initial access matrix.

Step 14: I identified possible production improvements, such as using AWS Organizations, Service Control Policies, and IAM Identity Center.

Step 15: I presented the access model as an HR/IT security policy recommendation.

3. Key AWS Concepts Learned

- **Least Privilege Principle:** I learned the importance of granting only the permissions required to perform a job and nothing more. This is the foundation of AWS security.
 - **IAM Policies:** I got hands-on experience writing IAM policies in JSON format, defining allowed and denied actions on specific resources.
 - **Role-Based Access Control (RBAC):** I understood how assigning permissions based on job roles, rather than individual users, simplifies management and improves security.
 - **Access Matrix:** I realized that creating a visual tool to map responsibilities to required permissions is extremely helpful before writing any policies.
 - **Conditional Policies:** I learned how to use Condition blocks to restrict access based on environment tags like dev, test, or prod.
 - **Denied Actions Testing:** I practiced using the AWS CLI to confirm that unauthorized actions were properly blocked, which is a great way to validate IAM policies.
-

4. Challenges Faced

- **Defining Granular Permissions:** The biggest challenge was deciding exactly which actions each role should have. For example, I had to think carefully about whether a Developer should be able to start and stop EC2 instances or just describe them. I decided to give Developers read-only access to EC2 and write access only to dev S3 buckets.
- **Conditional Access:** Using Condition tags like `ec2:ResourceTag/Environment : dev` was tricky to get right. I

had to test thoroughly to ensure the conditions worked as expected.

- **Testing Denied Actions:** It was confusing to test denied actions because I had to use specific instance IDs to get a proper AccessDenied error. I had to be very careful with the commands.
- **Policy Attachment Time:** After attaching a policy, there was a slight delay before the permissions propagated, which caused some early test failures. I learned to wait a few seconds before testing.

5. Solutions Implemented

- **Granular Policy Design:** I created separate policies for each role and used Condition blocks to restrict access by environment tags. For example, the DevPolicy only allows ec2:StartInstances if the instance has the tag Environment=dev.
- **Testing Strategy:** I used real instance IDs to test denied actions. This gave me actual AccessDenied errors, which confirmed that the policies were working correctly.
- **Patience with Propagation:** I learned to wait about 10 to 15 seconds after attaching a policy before running tests to avoid false errors.
- **Using Profiles:** For testing different users, I used the --profile flag to switch contexts easily. This made the testing process much smoother.

6. Security Considerations

- **Least Privilege is Key:** Every user was given only the minimum permissions required for their role. This reduces the attack surface significantly.
- **Environment Isolation:** Developers could only affect dev resources, QA could only affect test resources, and only Operations could touch production resources.

- **No Admin Access:** No user was given AdministratorAccess except for the task3-iam-user, which was a learning scenario.
 - **MFA Recommendation:** I strongly recommended enabling MFA for all users, especially those with production access, to add an extra layer of security.
 - **Audit Trail:** I recommended enabling CloudTrail to log all API calls for security auditing and compliance.
 - **Credential Rotation:** I documented the importance of rotating access keys every 90 days to minimize the risk of compromised credentials.
-

7. Results and Observations

Allowed Actions

Both Developer and QA users were able to successfully describe EC2 instances. The commands ran without any errors, confirming that the policies granted the necessary read-only access.

Denied Actions

When I tried to start EC2 instances using both the Developer and QA users, I received AccessDenied errors. This confirmed that the policies were correctly blocking unauthorized actions.

Observations

The access matrix was extremely useful for visualizing permissions before creating policies. Testing both allowed and denied actions gave me confidence that the policies were correctly implemented.

The Condition block in policies is a powerful way to restrict access based on tags, which is essential for multi-environment setups.

8. Possible Improvements for Production

If this setup were to be deployed in a production environment, I would recommend the following improvements:

- **AWS Organizations and SCPs:** This would provide centralized governance and guardrails at the account level.
 - **IAM Identity Center (SSO):** This would enable centralized user management and single sign-on for AWS accounts.
 - **MFA Enforcement:** All users should be required to have MFA enabled for their accounts.
 - **CloudTrail and Config:** These should be enabled for continuous compliance monitoring and auditing.
 - **Permission Boundaries:** These should be used to limit the maximum permissions a user can have.
 - **Temporary Credentials (STS):** AWS STS should be used for time-bound access instead of permanent credentials.
 - **Automated Access Reviews:** I would schedule quarterly reviews of all IAM permissions to remove unnecessary access.
-

9. HR/IT Security Policy Recommendation

Based on this exercise, I prepared a security policy recommendation that can be presented to HR and IT teams:

- **Policy Statement:** Access to AWS resources must be granted based on the principle of least privilege. All users must have unique accounts with MFA enabled.
- **Role-Based Access:** Developers should have access to the dev environment only. QA Engineers should have access to the test environment only. Operations Engineers should have access to the production environment only. HR/Admin Users should have access to IAM user management and billing only.
- **Credential Management:** Access keys must be rotated every 90 days. Passwords must be at least 12 characters long and include special characters. MFA is mandatory for all users.
- **Auditing:** All API calls must be logged via CloudTrail. IAM permissions must be reviewed quarterly.

- **Termination Procedure:** All access must be revoked within 24 hours of an employee's termination.
 - **Incident Response:** Any suspected breach must be reported immediately to the security team.
-

10. Conclusion

This task gave me practical experience in designing and implementing a secure, role-based IAM access structure using AWS IAM. By creating separate policies for each role, using environment tags for isolation, and testing both allowed and denied actions, I was able to validate that the access model works as intended. The key takeaway is that IAM is the foundation of AWS security—when properly configured, it prevents unauthorized access and minimizes the risk of data breaches.